

Graphs

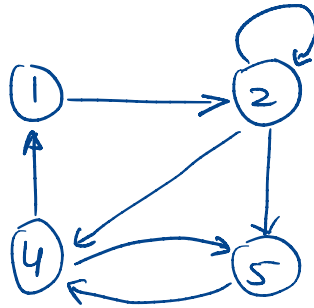
Graphs $\begin{cases} \text{Directed} : \text{edges have a direction} \\ \text{Undirected} : \text{edges are undirected} \end{cases}$

A directed graph (or a digraph) G is a pair (V, E) where V is a finite set, and E is a binary relation on V .

$\rightarrow V$ is called the vertex set of G (and its elements are called vertices)

$\rightarrow E$ is called the edge set of G (and its elements are called edges)

Example :



$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2), (2, 2), (2, 4), (2, 5), (4, 1), (4, 5), (5, 4), (6, 3)\}$$

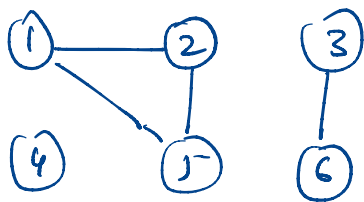
Note : Self loops are permissible

— In an undirected graph $G = (V, E)$, the edge set consists of unordered pair of vertices; rather than ordered.

That is, an edge is a set $\{u, v\}$ where $u, v \in V$ and $u \neq v \Rightarrow (u, v) \& (v, u)$ are the same!

↪ self loops are forbidden.

Example 6



$V = \{1, 2, 3, 4, 5, 6\}$

$E = \{ (1, 2), (1, 5), (2, 5), (3, 6) \}$

— In a digraph, if (u, v) is an edge then (u, v) is incident from or leaves vertex u , and is incident to or enters vertex v .

- If (u, v) is an edge in graph $G(V, E)$ then we say that vertex v is adjacent to vertex u .

→ for undirected graphs, adjacency relationship is symmetric

→ In a digraph if v is adjacent to u , then it is written as $u \rightarrow v$

- The degree of a vertex in an undirected graph is the # of edges incident on it

→ A vertex with a degree of 0 is called an isolated vertex.

- In a directed graph, we define:

in-degree : # of incoming edges

out-degree : # of outgoing edges

degree : in-degree + out-degree

Representation of graphs:

Two standard ways

- Adjacency list
- Adjacency matrix

Both directed and undirected graphs

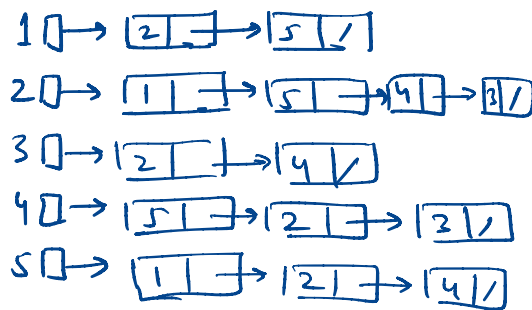
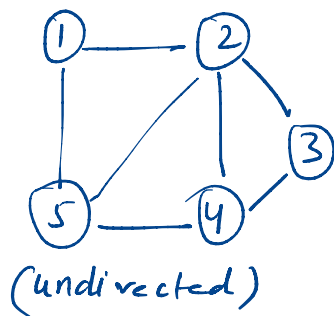
→ Adjacency list: provides a compact representation for sparse graphs — for which $|E|$ is much less than $|V|^2$

→ Adjacency matrix: good for dense graphs — for which $|E|$ is close to $|V|^2$.

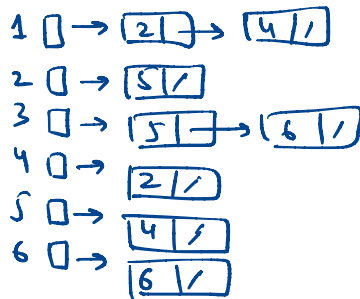
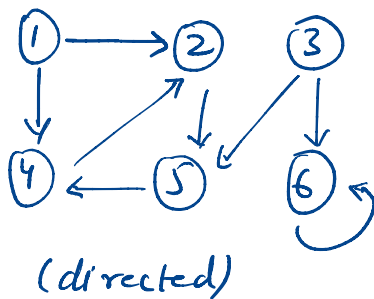
- Adjacency list: representation of a graph $G=(V,E)$ that consists of an array Adj of $|V|$ lists, one for each vertex in V . For each $u \in V$, the adjacency list $\text{Adj}[u]$ contains all vertices v such that there is an edge $(u,v) \in E$. → unordered!

→ i.e., $\text{Adj}[u]$ contains all vertices adjacent to u in G .

Example 2

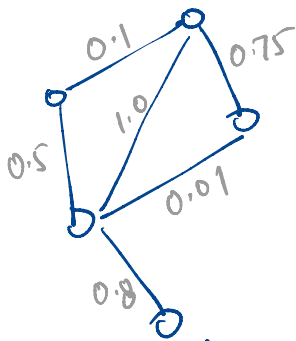


of edges stored = $2|E|$



— $\Theta(|E| + |V|)$ memory requirement

— Finding an edge (u, v) takes $\Theta(|E|)$ in the worst case.



a weighted graph

→ Can also be easily modified to represent weighted graphs

$w: E \rightarrow \mathbb{R}$, by storing weight w for edge (u, v) .

Adjacency Matrix representation :

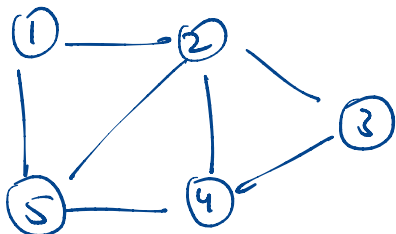
Assumes vertices are numbered $1, 2, \dots, |V|$.

The adjacency matrix is a $|V| \times |V|$ matrix

$A = (a_{ij})$ such that

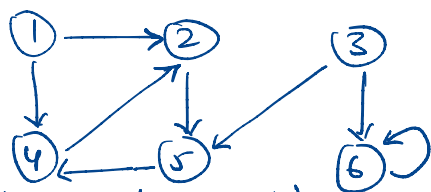
$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

Examples :



(undirected graph)

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0



(directed graph)

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

- Requires $O(|V|^2)$ memory independent of # of edges
- For undirected graph, A is symmetric. i.e, $A = A^T$
- Can be modified to store weighted graph.

in which case, $a_{ij} = \begin{cases} W(i, j) & \text{if } (i, j) \in E \\ NIL & \text{otherwise} \end{cases}$

Breadth First Search (BFS) :

- Given a graph $G=(V,E)$, and a source vertex s , BFS systematically explores the edges of G to "discover" every vertex that is reachable from s .
 - works for both directed & undirected graphs
 - It computes distance from s to each reachable vertex in G
- Also produces a Breadth-First-Tree with root s that contains all reachable vertices.
- For any vertex v reachable from s , the simple path in the breadth-first-tree from s to v corresponds to the shortest path from s to v in G (i.e., a path containing least # of edges)

- The BFS algorithm colors each vertex:
white, Gray, & Black

↳ Initially all vertices are white

↳ A discovered vertex becomes gray

↳ A gray vertex whose edges are explored turns black.

- All gray vertices are kept in a FIFO queue

- The BFS procedure assumes graph $G = (V, E)$ represented using adjacency lists, the queue is denoted by Q , and it attaches attributes to v :

↳ $v.color$: WHITE, GRAY, or BLACK

↳ $v.d$: distance from vertex s to v

↳ $v.\pi$: is v 's predecessor in BF tree

If v has no predecessor (v is source or undiscovered)
then $v.\pi = NIL$

BFS(G, s)

for each vertex $u \in G.V - \{s\}$

| $u.color = WHITE$
| $u.d = \infty$
| $u.\pi = NIL$

$s.color = GRAY$

$s.d = 0$

$s.\pi = NIL$

$Q = \emptyset$

Enqueue(Q, s)

while $Q \neq \emptyset$

| $u = \text{Dequeue}(Q)$

| for each vertex v in $G.Adj[u]$

| | if $v.color == WHITE$

| | | $v.color = GRAY$

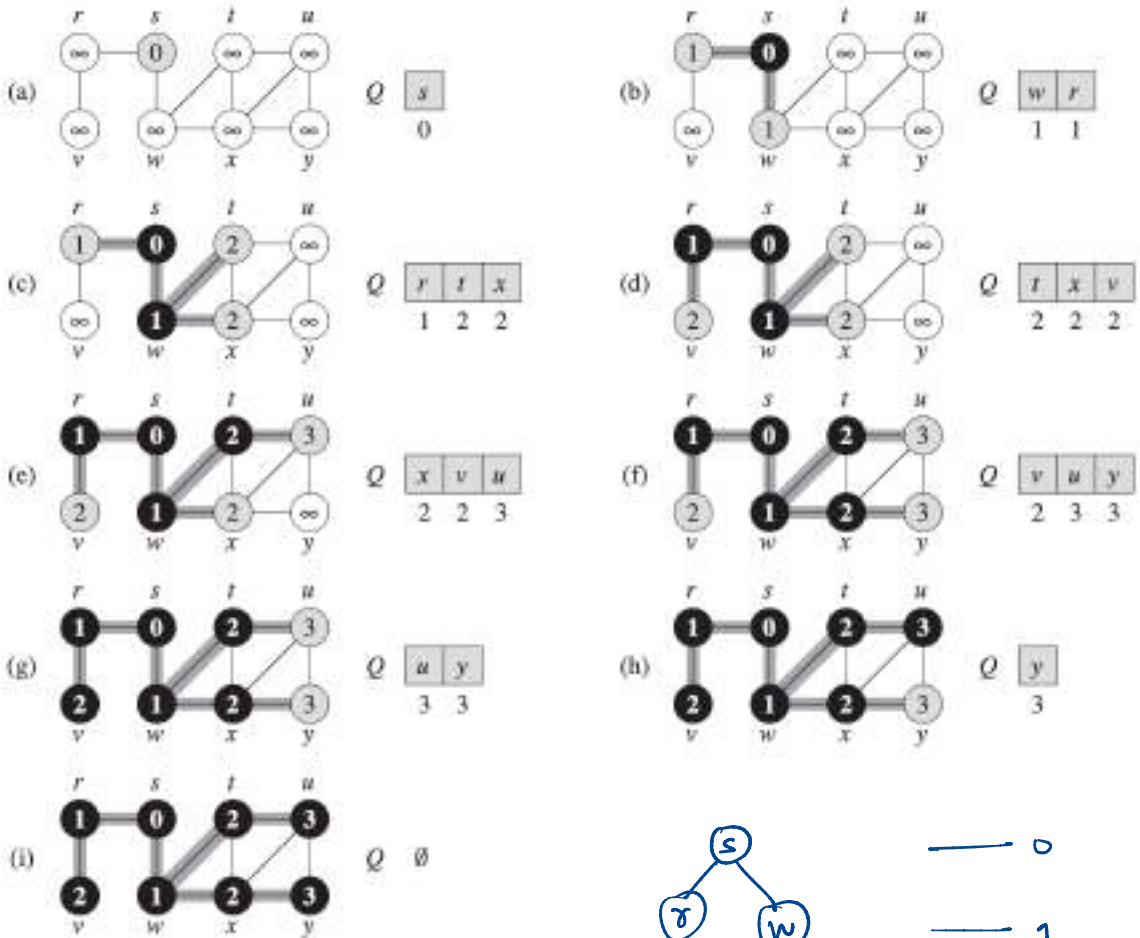
| | | $v.d = u.d + 1$

| | | $v.\pi = u$

| | | Enqueue(Q, v)

| $u.color = BLACK$

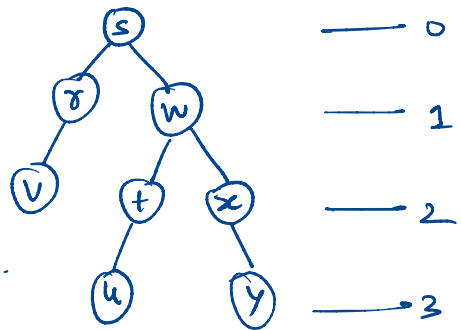
Example 8



Analysis: Enqueue, degree — $O(|V|)$
 ↳ total time for queue ops.
 Adj list of each vertex is scanned only once.

$$\Rightarrow \theta(|E|)$$

$$\Rightarrow \text{total time: } O(|V| + |E|) + \text{overhead } O(|V|)$$



Breadth-first-tree.

Depth first Search (DFS):

- Search deeper in the graph whenever possible
- Explore edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "back tracks" to explore edges leaving the vertex from which v was discovered.

Predecessor Subgraph: $G_\pi = (V, E_\pi)$, where G_π is a subgraph of original graph G

$$E_\pi = \{ (v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL} \}$$

- DFS forms a depth-first forest comprising of several depth-first trees.
- The edges in E_π are tree edges

- Depth first search also timestamps each vertex

→ Two time stamps:

① $v.d$ when v is first discovered

⇒ grayed out

② $v.f$ when search finishes

examining v 's adj. list

⇒ blackened

for every vertex u , $u.d < u.f$

- Before time $u.d$ → vertex u is colored WHITE

- Between $u.d$ & $u.f$ → vertex u is " GRAY

- After $u.f$ → " " " " BLACK

DFS (G)

for each vertex $u \in G.V$

| $u.color = WHITE$

| $u.\pi = NIL$

time = 0 // global variable

for each vertex $u \in G.V$

| if $u.color == WHITE$

| | DFS-VISIT (G, u)

DFS-VISIT (G, u)

time = time + 1

$u.d = \text{time}$ // white vertex u discovered

$u.\text{color} = \text{GRAY}$

for each vertex v in $G.\text{Adj}[u]$

 if $v.\text{color} == \text{WHITE}$

$v.\pi = u$

 DFS-VISIT (G, v)

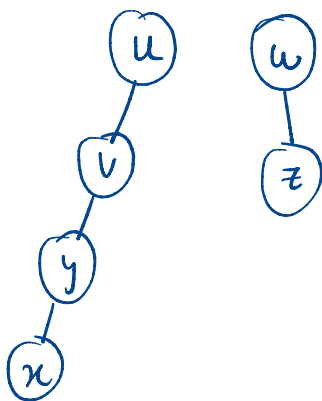
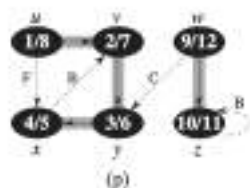
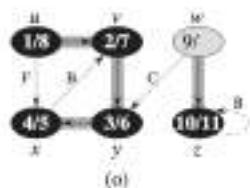
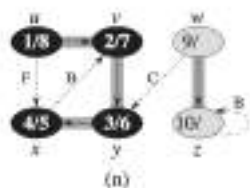
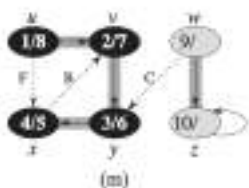
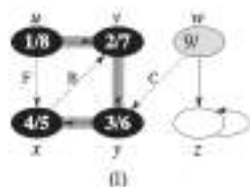
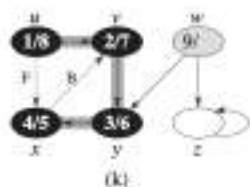
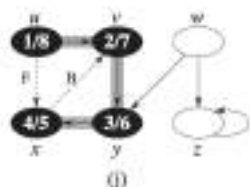
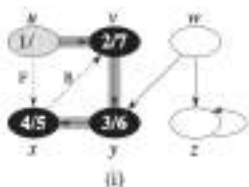
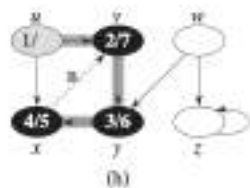
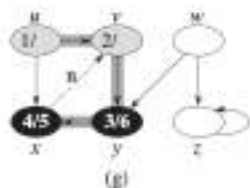
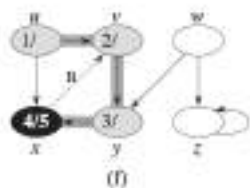
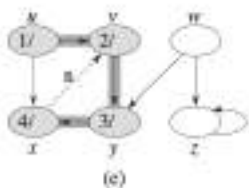
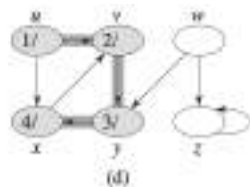
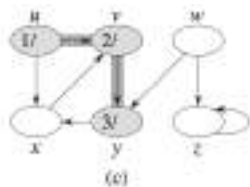
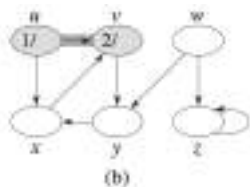
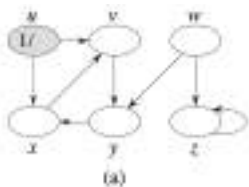
time = time + 1

$u.f = \text{time}$

$u.\text{color} = \text{BLACK}$

-
- Every call of DFS-VISIT(G, u) makes u the root of a new tree in the depth-first forest.
 - Results of depth-first search may depend on the order of selecting vertex u for v & on the order of vertices in the adjacency lists.

- Complexity : $\Theta(|V| + |E|)$



Classification of edges in G

↳ DFS classifies the graph edges into 4 types:

1. Tree edges: Edges in G_π . Edge (u, v) is a tree-edge if v was first discovered by exploring an edge (u, v) .
2. Back edges: are those edges (u, v) connecting a vertex u to an ancestor in a DF tree.
3. Forward edges: are those non-tree edges (u, v) connecting a vertex u to a proper descendent v in a DF tree.
4. Cross edges: all other edges. They can go between vertices of the same DF tree (as long as one vertex is not an ancestor of the other), or between different DF trees.

- The DFS algorithm has sufficient information to classify some edges during the run. While we explore an edge (u, v) , the color of v indicates:

WHITE $\rightarrow$ tree edge

GRAY $\rightarrow$ back edge

BLACK $\rightarrow$ forward edge or cross edge
 $\swarrow$
if $u.d < v.d$ if $u.d > v.d$

- In a depth-first search of an undirected graph, every edge is either a tree-edge or a back-edge.

- Applications $\begin{cases} \rightarrow \text{Cycle detection} \\ \rightarrow \text{Topological sorting} \end{cases}$

cycle detection: A directed graph is acyclic iff a depth-first search yields no "Back" edges.

Topological Sorting :

Given a Directed Acyclic Graph (DAG)

→ a directed graph
with no cycles

$G(V, E)$, a topological sort is a linear ordering of all its vertices s.t. if G contains an edge (u, v) then u appears before v in the ordering.

Topological-Sort (G)

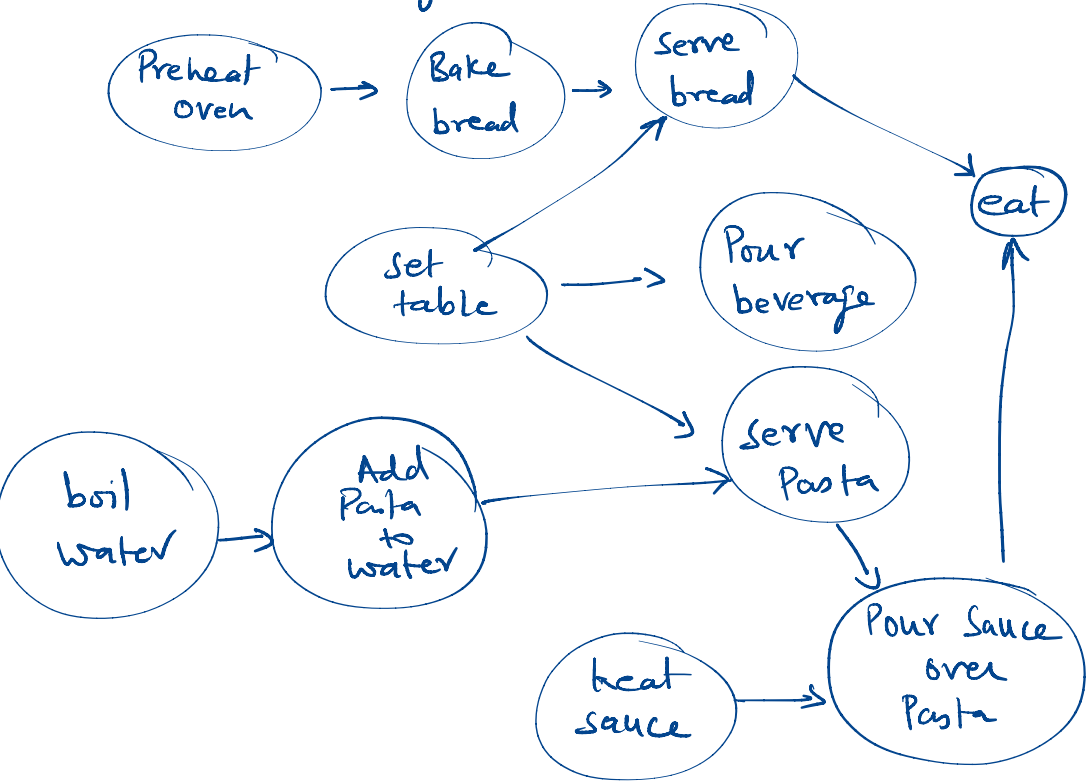
call DFS (G) to compute finish times $v.f$
of each vertex $v \in V$

↙ As each vertex is finished, insert it onto
the front of a linked list

$\Theta(|V| + |E|)$

Return the linked list of vertices

Example : Cooky Pasta as a DAG



↓ Topological sort

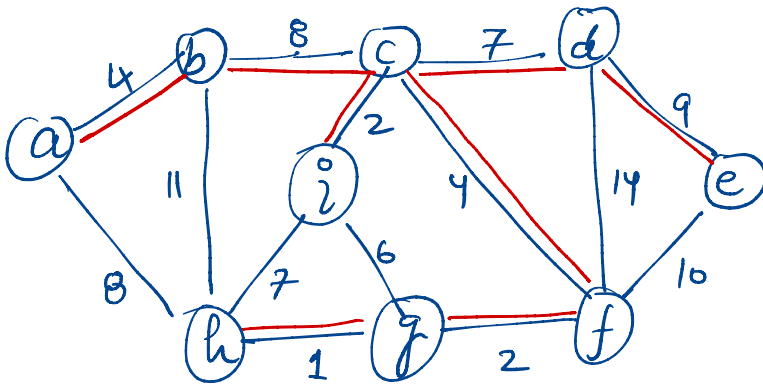


- Preheat oven
- set table
- boil water
- bake bread
- Add pasta to water
- Pour beverage
- Serve bread
- heat sauce
- Serve Pasta
- Pour sauce over Pasta
- eat

Minimum Spanning Trees

→ Given an undirected graph $G = (V, E)$ where every edge $(u, v) \in E$ has a weight $w(u, v)$, we wish to find an acyclic subset $T \subseteq E$ that connects all vertices & whose total weight $w(T) = \sum_{(u, v) \in T} w(u, v)$ is minimized.

- Since T is acyclic & connects all vertices, it forms a tree
- Such a tree is called the Minimum Spanning Tree



Algorithms $\rightarrow$ Kruskal's : $O(|E| \lg |V|)$ with a binary heap
 $\rightarrow$ Prim's : $O(|E| \lg |V|)$ with a binary heap
Greedy algorithm $\rightarrow O(|E| + |V| \lg |V|)$ with a Fibonacci heap
 $\rightarrow$ good if $|V| \ll |E|$

An algorithm that makes a choice that looks best at the moment
 $\Rightarrow$ a locally optimal choice

$\hookrightarrow$ Does not guarantee a globally optimal solution.

- General Strategy to find an MST

for a graph $G=(V,E)$ with a weight function

$w: E \mapsto \mathbb{R}$, a general greedy iterative approach is:

$\rightarrow$ Prior to each iteration, A is a subset of some MST

$\rightarrow$ At each step, we determine a edge (u,v) that can be added safely to A without violating the invariant —

$A \cup \{(u,v)\}$ is a subset of MST